

Project Report: Password Helper and Analyzer

General Overview

This project is designed to help users generate secure passwords and analyze their strength. It consists of multiple Java classes that work together to create strong passwords and assess their security level. The project aims to enhance password security by providing users with guidance on password strength and suggesting improvements where necessary. The primary objectives include:

- Generating random yet secure passwords.
- Evaluating password strength based on various security criteria.
- Providing feedback to users for improving password security.

Code Functionality

The project consists of three main Java classes: `Main.java`, `PasswordHelper.java`, and `PasswordAnalyzer.java`. Below is a detailed breakdown of their functionality:

1. `Main.java`

- This serves as the entry point of the program.
- It manages user interaction, allowing users to either generate a password or analyze an existing one.
- The program prompts users for input and then invokes the appropriate methods from `PasswordHelper` and `PasswordAnalyzer`.
- Based on the user's choice, it either generates a new password or evaluates an entered password.
- The results are displayed to the user, providing necessary feedback on password strength.

```
3 import java.util.Scanner;
4
5 public class Main {
6
7     public static void main(String[] args) {
8         Scanner scanner = new Scanner(System.in);
9
10        System.out.print("Enter Your Email: ");
11        String email = scanner.nextLine();
12
13        System.out.print("Enter Your Password: ");
14        System.out.println("The Recommended Password Must:\n");
15        + "~ Be Between 8-12 Characters.\n"
16        + "~ Contain At Least One UpperCase And One LowerCase Letter.\n"
17        + "~ Include A Number And A Special Symbol.\n"
18        + "~ Not Contain Common Words.\n"
19        + "~ Not Have Sequential Or Repeated Characters.\n"
20        + "~ Not Be Similar To The Email.");
21
22        String password = scanner.nextLine();
23
24        if (PasswordAnalyzer.isValidPassword(password, email)) {
25            System.out.println("Your Password Is Valid And Strong!");
26        } else {
27            System.out.println("Your Password Is Invalid And Weak!");
28            String suggestedPassword = PasswordHelper.generateValidPassword();
29            System.out.println("The Suggested Password is: " + suggestedPassword);
30        }
31
32        scanner.close();
33    }
34 }
```

2. PasswordHelper.java

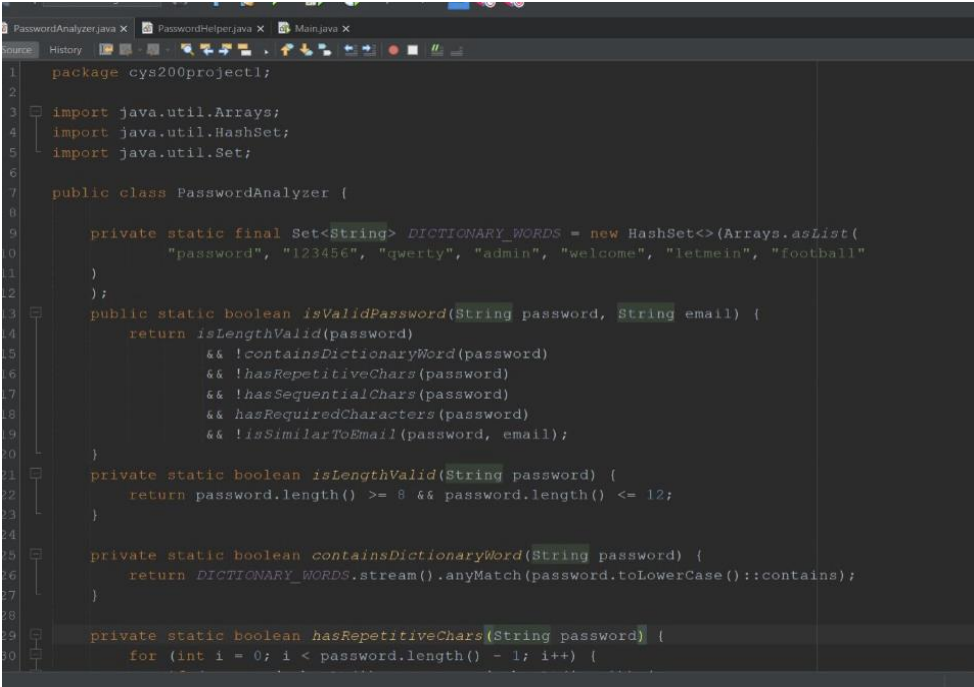
- This class is responsible for generating secure passwords based on defined security parameters.
- It creates passwords that include uppercase letters, lowercase letters, numbers, and special characters to enhance security.
- The method uses a randomization technique to ensure that each generated password is unique and difficult to guess.
- It may allow customization options such as password length and character inclusion preferences.
- Example logic in this class could involve using Java's Random class or SecureRandom for enhanced security.

```
1 package cys200project1;
2
3 import java.security.SecureRandom;
4
5 public class PasswordHelper {
6     private static final String UPPERCASE_LETTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
7     private static final String LOWERCASE_LETTERS = "abcdefghijklmnopqrstuvwxyz";
8     private static final String NUMBERS = "0123456789";
9     private static final String SPECIAL_CHARACTERS = "!@#$%^&*()-+=<>?";
10    private static final String ALL_CHARACTERS = UPPERCASE_LETTERS + LOWERCASE_LETTERS + NUMBERS + SPECIAL_CHARACTERS;
11    private static final SecureRandom RANDOM_GENERATOR = new SecureRandom();
12    public static String generateValidPassword() {
13        String suggestedPassword;
14        do {
15            suggestedPassword = createRandomPassword();
16        } while (!PasswordAnalyzer.isValidPassword(suggestedPassword, "")); // Ensuring it meets all conditions
17        return suggestedPassword;
18    }
19
20    private static String createRandomPassword() {
21        int passwordLength = RANDOM_GENERATOR.nextInt(5) + 8; // Random length between 8 and 12
22        StringBuilder password = new StringBuilder();
23
24        password.append(getRandomCharacter(UPPERCASE_LETTERS));
25        password.append(getRandomCharacter(LOWERCASE_LETTERS));
26        password.append(getRandomCharacter(NUMBERS));
27        password.append(getRandomCharacter(SPECIAL_CHARACTERS));
28
29        while (password.length() < passwordLength) {
30            password.append(getRandomCharacter(ALL_CHARACTERS));
31        }
32    }
33}
```

```
24 password.append(getRandomCharacter(UPPERCASE_LETTERS));
25 password.append(getRandomCharacter(LOWERCASE_LETTERS));
26 password.append(getRandomCharacter(NUMBERS));
27 password.append(getRandomCharacter(SPECIAL_CHARACTERS));
28
29 while (password.length() < passwordLength) {
30     password.append(getRandomCharacter(ALL_CHARACTERS));
31 }
32
33 return shufflePassword(password.toString());
34 }
35
36 private static char getRandomCharacter(String characterSet) {
37     return characterSet.charAt(RANDOM_GENERATOR.nextInt(characterSet.length()));
38 }
39
40 private static String shufflePassword(String password) {
41     char[] characters = password.toCharArray();
42     for (int i = characters.length - 1; i > 0; i--) {
43         int randomIndex = RANDOM_GENERATOR.nextInt(i + 1);
44         char temp = characters[randomIndex];
45         characters[randomIndex] = characters[i];
46         characters[i] = temp;
47     }
48     return new String(characters);
49 }
50 }
51
```

3. PasswordAnalyzer.java

- This class assesses the strength of a given password based on various security criteria.
- The analysis includes checking:
 - The length of the password (longer passwords are generally more secure).
 - The presence of uppercase and lowercase letters, numbers, and special characters.
 - Whether the password contains common words or predictable patterns.
- The program then categorizes the password into different strength levels, such as:
 - Weak: If it lacks diversity and is too short.
 - Moderate: If it meets some criteria but could be stronger.
 - Strong: If it meets all security recommendations.
- It provides feedback to the user, suggesting how to improve the password if necessary.



```
1 package cys200project1;
2
3 import java.util.Arrays;
4 import java.util.HashSet;
5 import java.util.Set;
6
7 public class PasswordAnalyzer {
8
9     private static final Set<String> DICTIONARY_WORDS = new HashSet<>(Arrays.asList(
10         "password", "123456", "qwerty", "admin", "welcome", "letmein", "football"
11     ));
12
13     public static boolean isValidPassword(String password, String email) {
14         return isLengthValid(password)
15             && !containsDictionaryWord(password)
16             && !hasRepetitiveChars(password)
17             && !hasSequentialChars(password)
18             && hasRequiredCharacters(password)
19             && !isSimilarToEmail(password, email);
20     }
21
22     private static boolean isLengthValid(String password) {
23         return password.length() >= 8 && password.length() <= 12;
24     }
25
26     private static boolean containsDictionaryWord(String password) {
27         return DICTIONARY_WORDS.stream().anyMatch(password.toLowerCase()::contains);
28     }
29
30     private static boolean hasRepetitiveChars(String password) {
31         for (int i = 0; i < password.length() - 1; i++) {
```

```
PasswordAnalyzer.java x PasswordHelper.java x Main.java x
Source History
1 if (password.charAt(i) == password.charAt(i + 1)) {
2     return true;
3 }
4 }
5 return false;
6 }
7
8 private static boolean hasSequentialChars(String password) {
9     for (int i = 0; i < password.length() - 1; i++) {
10         if (password.charAt(i) + 1 == password.charAt(i + 1)) {
11             return true;
12         }
13     }
14     return false;
15 }
16
17 private static boolean hasRequiredCharacters(String password) {
18     boolean hasUpper = false, hasLower = false, hasDigit = false, hasSymbol = false;
19
20     for (char ch : password.toCharArray()) {
21         if (Character.isUpperCase(ch)) {
22             hasUpper = true;
23         } else if (Character.isLowerCase(ch)) {
24             hasLower = true;
25         } else if (Character.isDigit(ch)) {
26             hasDigit = true;
27         } else {
28             hasSymbol = true;
29         }
30     }
31 }
```

```
PasswordAnalyzer.java x PasswordHelper.java x Main.java x
Source History
54     hasLower = true;
55 } else if (Character.isDigit(ch)) {
56     hasDigit = true;
57 } else {
58     hasSymbol = true;
59 }
60 }
61
62 return hasUpper && hasLower && hasDigit && hasSymbol;
63 }
64
65 private static boolean isSimilarToEmail(String password, String email) {
66     if (email == null || email.isEmpty()) {
67         return false;
68     }
69     String emailLocalPart = email.split("@")[0];
70     return password.toLowerCase().contains(emailLocalPart.toLowerCase());
71 }
72 }
73 }
```

Output:

```
Output - cys200project1 (run) X
Run:
Enter Your Email: layan4@gamil.com
Enter Your Password: The Recommended Password Must:
- Be Between 8-12 Characters.
- Contain At Least One UpperCase And One LowerCase Letter.
- Include A Number And A Special Symbol.
- Not Contain Common Words.
- Not Have Sequential Or Repeated Characters.
- Not Be Similar To The Email.
layan@@a
Your Password Is Invalid And Weak!
The Suggested Password is: ^mLX^6rxV
BUILD SUCCESSFUL (total time: 25 seconds)
building cys200project1 (run).
```

```
put - cys200project1 (run) X
Run:
Enter Your Email: layan4@gmail.com
Enter Your Password: The Recommended Password Must:
- Be Between 8-12 Characters.
- Contain At Least One UpperCase And One LowerCase Letter.
- Include A Number And A Special Symbol.
- Not Contain Common Words.
- Not Have Sequential Or Repeated Characters.
- Not Be Similar To The Email.
P2M_Log-$
Your Password Is Valid And Strong!
BUILD SUCCESSFUL (total time: 19 seconds)
200project1 (run).
```

Conclusion

This project provides a valuable tool for users looking to improve their password security. By combining password generation and strength analysis, it helps users create and maintain robust passwords that enhance their digital security. The structured approach in Java ensures scalability and ease of use, making it a practical solution for personal and organizational password management.

